

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего профессионального образования
«Вятский государственный университет»
(ФГБОУ ВПО «ВятГУ»)
Факультет автоматики и вычислительной техники
Кафедра электронных вычислительных машин

Методические указания
к самостоятельным и лабораторным работам
по дисциплине «Базы данных»

Лабораторная работа №1.
Основы DDL-запросов в PostgreSQL

ВятГУ, 2019

1. Цели лабораторной работы:

- познакомиться со схемами, пользователями и ролями в PostgreSQL;
- познакомиться с типами данных в PostgreSQL;
- освоить основные варианты DDL-запросов в PostgreSQL;
- закрепить знания по проектированию структуры реляционной БД;
- создать рабочий материал для следующих лабораторных работ.

2. Задание на лабораторную работу:

Нужно выполнить следующие шаги.

2.1. Разработать структуру базы данных на любую выбранную тему.

Структура должна отвечать следующим условиям (это важно для следующих лабораторных работ):

- должно быть не меньше пяти таблиц;
- хотя бы одна таблица должна содержать колонку с числовыми данными;
- структура БД не должна быть связью много-ко-многим (добавляем промежуточную таблицу для связей).

2.2. Создать нового пользователя и пустую БД. Подключиться к созданной БД.

2.3. Написать и выполнить SQL-скрипт, создающий таблицы согласно разработанной структуре БД. Созданный в п.2 пользователь должен иметь все права на созданные объекты. В этом же скрипте должны создаваться нужные ограничения и индексы:

- обязательно должны быть созданы внешние ключи для поддержания ссылочной целостности;
- желательно должны быть проставлены ограничения и уникальные индексы для поддержания консистентности данных (например, если в таблице есть столбец с температурой воздуха за окном, стоит ограничить возможный промежуток значений);
- желательно должны быть проставлены индексы для производительности там, где они могут помочь (из расчета, что созданная БД будет хранить достаточно большое количество данных).

3. Отчет по лабораторной работе должен содержать:

- ER-диаграмму для разработанной структуры БД;
- описание выбранной темы и краткое описание таблиц;
- SQL-скрипт из п.3 задания с комментариями.

4. Рекомендации по проектированию структуры БД

- начать проектирование можно сразу с даталогической модели;
- названия таблиц и столбцов должны быть на английском языке, не транслит;
- таблицы лучше называть именем существительным в единственном числе; исключение – для таблиц, название которых совпадает с ключевыми словами (вместо таблицы user – таблица users);
- в именах любых объектов (кроме переменных и параметров в PL/pgSQL, о котором рассказано в лабораторной работе №3) правильно использовать подчеркивания (good_name – правильно, badName – неправильно)
- важно помнить, что имена регистронезависимые;
- самый лучший тип для столбца с первичным ключом – bigserial; индекс по числовому столбцу работает быстрее всего, автоинкремент помогает поддерживать уникальность, а на размер столбца под первичный ключ лучше не скупиться – если значения smallserial или serial закончатся, это вызовет проблемы;
- соответственно, тип для столбцов для внешних ключей – bigint;
- столбец с первичным ключом лучше всего назвать просто id, так исчезает любая путаница; если, например, у таблицы customer столбец с первичным ключом будет называться customer_id или как-то иначе, его предназначение будет неочевидно;
- соответственно, столбец со внешним ключом лучше назвать как «имя_таблицы_id» (внешний ключ от таблицы orders в таблицу customer – customer_id), если оно не несет какого-либо дополнительного смысла; в таком случае суффикс «_id» все равно лучше использовать;
- столбцы типа boolean лучше называть с использованием префикса «is_» (иногда еще используется суффикс «_f»);
- важно помнить, что максимальная длина имени объекта в PostgreSQL (без имени схемы и т.д.) – 63 символа;
- ER-диаграмму можно сгенерировать автоматически, используя, например, средства DataGrip, DbVisualizer или выбрав PgAdmin в меню для БД опцию с ERD.

5. Ход выполнения лабораторной работы

5.1. Структура базы данных на любую выбранную тему

Выберем для структуры базы данных на тему «*Студенты*». Каждый студент имеет имя, фамилию и, возможно, отчество, так же адрес и курс обучения, группу, дисциплины, оценки за дисциплины. Так же дисциплины имеют свой тип, например, лекция, практическое занятие, лабораторная работа и так далее. Дисциплины как проходятся студентами, так и проводятся преподавателями.

Первой таблицей будет таблица «студенты», которая должна иметь: идентификатор, фамилию, имя, отчество, адрес, курс обучения и идентификатор группы студента.

Второй таблице будет «группы», так как сущности «студенты» и «группы» связаны связью многие-к-одному, т. е. у одной группы может быть много студентов и у многих студентом может быть одна группа, чтобы эти сущности были в 3-й нормальной форме, следует разделить их на разные таблицы. (Более подробней о нормальных формах, вы узнаете на лекции). Таблица «группы» будет содержать следующие поля: идентификатор, название группы, количество студентов в группе.

Далее рассмотрим сущности «студенты» и «дисциплины». Данные сущности связаны связью многие-ко-многим, т. к. многие студенты могут иметь много дисциплин, и многие дисциплины могут изучаться многими студентами. Сущности со связью многие-ко-многим разбиваются на 3 таблицы (в 3 нормальной форме): 2 таблицы – сами сущности и 1 промежуточная таблица, которая хранит связи между таблицами и, возможно, еще какие-либо данные.

В итоге получаем таблицу «дисциплины» с полями: идентификатор, название дисциплины и количество часов на изучение данной дисциплины.

Промежуточную таблицы между студентами и дисциплинами назовем «прогресс» с полями: ид студента, ид предмета, оценка за предмет.

Остается рассмотреть сущности «преподаватели» и «дисциплины». Данные сущности имеют связь многие-ко-многим, требуется 3 таблицы, таблица «дисциплины» уже есть, остается описать таблицу «преподаватели», которая будет иметь поля: идентификатор, ФИО преподавателя, его ученую степень и номер мобильного телефона, и промежуточную таблицу, которая будет хранить помимо идентификаторов дисциплин и преподавателей еще и тип дисциплины, и назовем данную таблицу «тип занятия».

ER-диаграмму для отчета следует выполнить самостоятельно. (Подробнее о том, как строить ER-диаграммы, будет на лекциях).

5.1 Создание нового пользователя и пустой БД. Подключение к созданной БД.

Создание базы данных.

Любой сервер PostgreSQL может обслуживать несколько БД сразу. По умолчанию после установки он содержит только одну базу данных – postgres, которая содержит системную информацию.

Подробнее о базовых командах создания и управления настройками БД в целом можно прочитать в документации (стр. 1319; <https://postgrespro.ru/docs/postgresql/9.6/sql-createdatabase.html>).

Все команды будем выполнять в pgAdmin III в инструменте запросов и PSQL Console. Перейти в инструмент запросов можно «Инструменты/инструмент запросов» (Ctrl + E). А в PSQL Console – Плагины/PSQL Console

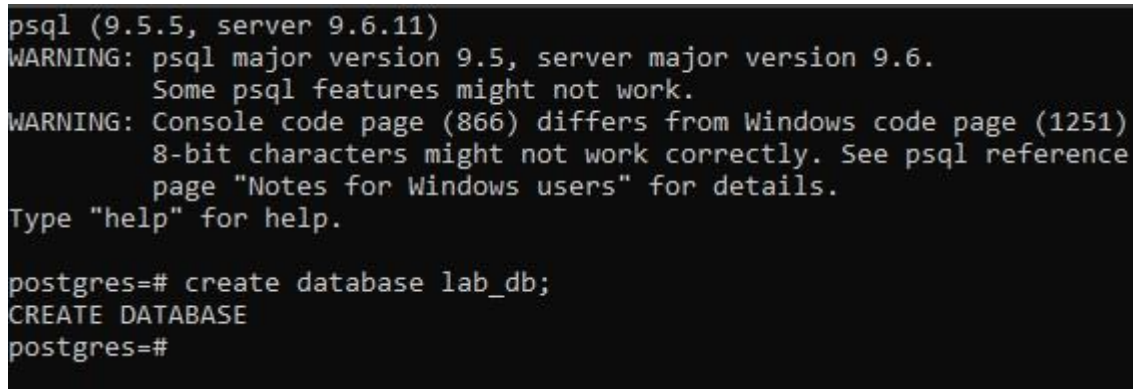
Ниже показаны примеры команд для создания и удаления БД.

```
-- Создать БД lab_db
create database lab_db;

-- Две команды ниже эквиваленты и создают БД lab_db, назначая владельцем
пользователя lab_test
-- Важно, если lab_test должен быть владельцем, а команда выполняется из-под
другого пользователя
create database lab_db owner lab_test;
create database lab_db with owner lab_test;

-- Удалить БД lab_db и все её данные
drop database lab_db;
```

Создадим базу данных с названием «lab_db» с помощью команды в PSQL Console (Рис. 1). Чтобы можно было открыть PSQL Console, для начала следует выбрать базу данных, под которой мы зайдём, бд «postgres» создается по умолчанию, выделяем её, после чего можно открывать PSQL Console.



```
psql (9.5.5, server 9.6.11)
WARNING: psql major version 9.5, server major version 9.6.
Some psql features might not work.
WARNING: Console code page (866) differs from Windows code page (1251)
8-bit characters might not work correctly. See psql reference
page "Notes for Windows users" for details.
Type "help" for help.

postgres=# create database lab_db;
CREATE DATABASE
postgres=#
```

Рис. 1 – Создание базы данных в psql.

Убедимся, что данная база данных создавалась, перейдем в pgAdmin и в браузере объектов, выберем созданный нами сервер (выделить сервер и нажать кнопку обновить выделенный объект (пкм/обновить или f5)), и увидим 2 базы данных: postgres, которая создается вместе с сервером и lab_db, которую мы создали (Рис 2.)

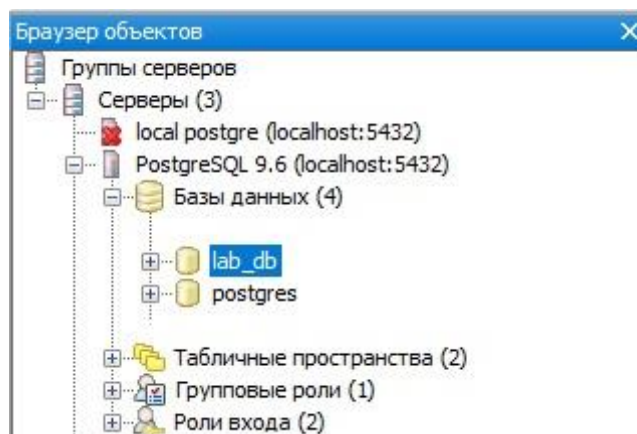


Рис. 2 Браузер объектов pgAdmin.

Немного о схемах.

В PostgreSQL база данных состоит из именованных схемы. Схемы, в свою очередь, содержат таблицы, функции, типы данных и другие объекты.

Чтобы обращаться к объектам внутри схемы, необходимо указывать их полное имя, состоящее из имени схемы и имени объекта, разделённых точкой:

```
test_schema.log -- Схема test_schema, таблица log
```

По умолчанию каждая БД содержит только одну схему – *public*. Если не указывать имя схемы перед именем объекта, автоматически будет подставляться *public*.

```
-- Следующие два запроса эквивалентны
select * from public.table1;
select * from table1;
```

После создания схемы все права на неё (и объекты внутри) имеет только владелец (тот, кто создал схему, если вручную при создании не был указан другой пользователь). Всем остальным пользователям необходимо выдать соответствующие права, если им нужен доступ к объектам внутри схемы.

По умолчанию все пользователи имеют полные права на схему *public*.

Несмотря на то, что можно разместить все объекты БД в схеме public, это считается плохим тоном.

Подробнее о схемах можно прочитать в документации (гл. 5.8, стр. 62; <https://postgrespro.ru/docs/postgresql/9.6/ddl-schemas>).

Ниже показаны примеры различных команд, связанных с созданием схем в PostgreSQL.

```
-- Создать схему test_schema
create schema test_schema;

-- Создать схему test_schema и назначить пользователя lab_test владельцем
-- Важно, если lab_test должен быть владельцем, а команда выполняется из-под
другого пользователя
create schema test_schema authorization lab_test;

-- Создать таблицу blah_blah в схеме test_schema
create table test_schema.blah_blah(
    ... -- Описание столбцов
);

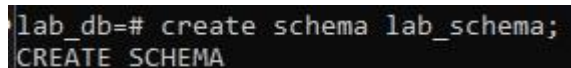
-- Удалить пустую схему test_schema (если в ней есть какие-либо объекты,
поднимется ошибка)
drop schema test_schema;

-- Удалить схему test_schema и все объекты внутри неё
drop schema test_schema cascade;
```

Создадим схему «lab_schema» с помощью команды в PSQL Console (Перед тем как открыть PSQL Console выделите базу данных, под которой будет выполнен заход, в нашем случае «lab_db») (Рис. 3).

Этот пункт не обязательный, но стоит учитывать, что при его не выполнении, вы будете работать в схеме public, а не lab_schema, так же все дальнейшие примеры приведены в схеме lab_schema.

```
create schema lab_schema;
```



```
lab_db=# create schema lab_schema;
CREATE SCHEMA
```

Рис. 3 – Создание схемы.

Убедимся, что схема появилась в базе данных «lab_db» (Рис. 4)



Рис. 4 – Список схем в базе данных.

Роли

Для разграничения доступа к серверу, базам данных и объектам внутри них используется ролевая система.

Роль можно рассматривать как пользователя базы данных или как группу пользователей, в зависимости от того, как роль настроена. Роли могут владеть объектами базы данных (например, таблицами и функциями) и выдавать другим ролям разрешения на доступ к этим объектам, управляя тем, кто имеет доступ и к каким объектам. Кроме того, можно предоставить одной роли членство в другой роли, таким образом одна роль может использовать привилегии других ролей.

Подробнее о ролях можно прочитать в документации (гл. 21, стр. 549; <https://postgrespro.ru/docs/postgresql/9.6/user-manag>).

Ниже показаны примеры различных команд, связанных с созданием и удалением ролей.

```
-- Две команды ниже аналогичны
-- В результате любой из них будет создан пользователь lab_test с паролем
lab_test
create user lab_test password 'lab_test';
create role lab_test password 'lab_test' login;

-- Две команды ниже аналогичны
-- Удалить созданную роль (пользователя)
drop user lab_test;
drop role lab_test;
```

Создадим роль через команду, описанную выше, и, используя, PSQL Console (Перед тем как открыть PSQL Console выделите базу данных, под которой будет выполнен заход, в нашем случае «lab_db»).

После выполнения команды убедимся, что роль успешно создана (Рис. 5), для этого выберете сервер в окне «Браузер объектов» в pgAdmin III, после чего обновите содержимое сервера (пкм/обновить или F5)



Рис. 5 – Список ролей на сервере.

4.2. Написание и выполнение SQL-скрипта, создающий таблицы согласно разработанной структуре БД.

Для написания SQL-скриптов используем «Инструмент запрос» и разработанную структуру БД в пункте 4.1.

Создадим таблицы «Студенты» и «Группы».

Создание и удаление таблиц.

Таблицы создаются командой *create table*. В рамках курса лабораторных работ рассматривается самый простой вариант этой команды.

Подробнее о полном синтаксисе команды *create table* можно прочитать в документации (стр. 1379; <https://postgrespro.ru/docs/postgresql/9.6/sql-createtable.html>).

Ниже показан пример использования команды *create table*.

```
-- Создает таблицу groups в схеме lab_schema
create table lab_schema.groups (
    id                bigserial primary key,
    name              varchar(30) not null,
    number_of_students integer not null check (number_of_students > 0),
);

-- Создает таблицу students в схеме lab_schema
create table lab_schema.students (
    id                bigserial primary key,
    first_name        varchar(30) not null,
    second_name       varchar(30) not null,
    middle_name       varchar(30),
    id_groups          bigint references lab_schema.groups(id),
    cell_phone        varchar(20),
    address            varchar(100),
    course            integer check (course > 0)
);
```

В качестве первичного ключа используется столбец *id*. Он имеет тип *bigserial*, что означает, что каждая новая строка будет получать новое уникальное значение *id* с помощью автоинкремента предыдущего выданного. Указание *primary key* после описания столбца показывает, что:

- столбец является первичным ключом;
- столбец не может содержать null;
- значение в столбце уникально.

Если описанию столбца указано *not null*, то он не может содержать null. Иначе столбец может содержать null.

Для изменения определения существующей таблицы используется команда *alter table*. Она позволяет добавлять и удалять колонки, переименовывать их, редактировать ограничения, триггеры, индексы и т.д.

Подробнее о полном синтаксисе команды *alter table* можно прочитать в документации (стр. 1253; <https://postgrespro.ru/docs/postgresql/9.6/sql-altertable.html>).

Ниже показаны примеры использования команды *alter table*.

```
-- Переименовать столбец address в hometown
alter table lab_schema.students rename address to hometown;

-- Установить значение по умолчанию для столбца hometown
alter table lab_schema.students alter hometown set default 'Киров';

-- Запретить столбцу hometown содержать null
-- Если в таблице уже есть строки, содержащие null в этом столбце,
-- обязательно указание
-- значения по умолчанию (команда выше)
alter table lab_schema.students alter hometown set not null;

-- Разрешить записывать null в столбец hometown
alter table lab_schema.students alter hometown drop not null;
```

Для удаления таблицы используется команда *drop table*:

```
-- Удалить таблицу lab_schema.students и все зависимые от неё объекты
-- Если слово cascade убрать (или вместо него написать restrict), то если
-- есть зависимые
-- объекты, поднимется ошибка
drop table lab_schema.students cascade;
```

Типы данных.

PostgreSQL предоставляет множество различных типов данных, как требуемых по стандарту SQL, так и специфичных для разных задач.

Далее рассмотрены наиболее часто применяющиеся типы данных, которые можно использовать для решения большинства задач.

В таблице 2.1 перечислены числовые типы данных общего назначения.

Таблица 4.1 – Числовые типы данных общего назначения

Имя	Псевдонимы	Описание
bigint	int8	знаковое целое из 8 байт
bigserial	serial8	восьмибайтное целое с автоувеличением
integer	int, int4	знаковое четырёхбайтное целое
numeric [(p, s)]	decimal [(p, s)]	вещественное число заданной точности
real	float4	число одинарной точности с плавающей точкой (4 байта)
smallint	int2	знаковое двухбайтное целое

smallserial	serial2	двухбайтное целое с автоувеличением
serial	serial4	четырёхбайтное целое с автоувеличением

В таблице 2.2 показаны символьные типы данных общего назначения.

Таблица 2.2 – Символьные типы данных общего назначения

Имя	Псевдонимы	Описание
character varying(n)	varchar(n)	строка ограниченной переменной длины
character(n)	char(n)	строка фиксированной длины, дополненная пробелами
text		строка неограниченной переменной длины

В таблице выше число n означает количество символов в кодировке БД.

В таблице 2.3 показаны типы даты/времени общего назначения (за исключением interval).

Таблица 2.3 – Типы даты/времени общего назначения

Имя	Размер	Описание	Мин. значение	Макс. значение	Точность
date	4 байта	дата (без времени суток)	4713 до н. э.	5874897 н. э.	1 день
time [(p)] [without time zone]	8 байт	время суток (без даты)	00:00:00	24:00:00	1 микросекунда / 14 цифр
time [(p)] with time zone	12 байт	только время суток (с часовым поясом)	00:00:00 +1459	24:00:00 -1459	1 микросекунда / 14 цифр

Тип time принимает необязательное значение точности p, определяющее, сколько знаков после запятой должно сохраняться в секундах. По умолчанию точность не ограничивается. В типах данных с указанием часового пояса дата/время хранится для часовой зоны UTC+00, а при чтении/записи данных учитывается либо явно заданный часовой пояс, либо часовой пояс клиента.

В таблице 2.4 показаны остальные типы данных общего назначения.

Таблица 2.4 – Остальные типы данных общего назначения

Имя	Псевдонимы	Описание
boolean	bool	логическое значение (true/false)
bytea		двоичные данные («массив байт»)

Тип `bytea` следует использовать, когда в колонке таблицы нужно хранить файл (изображение, документ и т.д.).

Во многих случаях полезными могут быть *перечисления*. Типы перечислений (`enum`) определяют статический упорядоченный набор значений, так же как и типы `enum`, существующие в ряде языков программирования. В качестве перечисления можно привести дни недели или набор состояний.

Ниже показана команда для создания перечисления типов занятий:

```
-- Создает тип-перечисление occupation_type_enum в схеме lab_schema
-- После этого можно указывать такой тип у колонок; попытка записать любое
-- значение, кроме
-- четырех указанных, вызовет ошибку
create type lab_schema.occupation_type_enum as enum('lecture', 'laboratory
work', 'practical lesson', 'seminar');
```

Важно, что нельзя сравнивать значения разных перечислений напрямую. Для этого можно, например, привести оба значения к типу `text` и после этого сравнивать.

Подробнее о всех существующих типах данных можно прочитать в документации (гл.8, стр. 105;

<https://postgrespro.ru/docs/postgresql/9.6/datatype.html>).

Ограничения

Типы данных сами по себе ограничивают множество данных, которые можно сохранить в таблице. Однако для многих задач такие ограничения слишком грубые. Например, столбец, содержащий цену товара, должен принимать только положительные значения. А столбец, содержащий код товара, скорее всего должен содержать уникальное значение.

Для этого существуют *ограничения (constraints)*. Они позволяют задавать особые условия для проверки данных, чтобы обеспечивать их *консистентность*.

Теоретически ограничения разделяются на ограничения столбца и таблицы. Ограничения столбца задаются только при создании таблицы и должны касаться только одного конкретного столбца. Ограничения таблицы могут быть созданы в любое время и могут относиться к группе столбцов.

Ограничение может как иметь имя, так и быть безымянным (тогда их имя автоматически назначается СУБД). Рекомендуется давать имя всем ограничениям, кроме тех, которые точно не могут измениться за период жизни таблицы.

Подробно про ограничения можно прочитать в документации (гл. 5.3, стр. 47; <https://postgrespro.ru/docs/postgresql/9.6/ddl-constraints>).

Далее перечислена наиболее важная информация о существующих типах ограничений.

Ограничения-проверки. В его определении указывается, что значение столбца должно удовлетворять определённому условию. Например, курс студента строго больше нуля.

Ниже показан пример команды для создания ограничений-проверок (Используется укороченный пример про студентов, взятый выше).

```
-- Создает таблицу students в схеме lab_schema, у которой есть одно
-- безымянное ограничение:
-- course > 0, описано на уровне столбца course (тем не менее, в PostgreSQL
-- это также ограничение таблицы)
create table lab_schema.students (
    id                bigserial primary key,
    first_name        varchar(30) not null,
    second_name       varchar(30) not null,
    middle_name       varchar(30),
    course            integer check (course > 0)
);
```

Любые ограничения можно создавать отдельно от команды создания таблицы с помощью команды *alter table* (см. 4.3).

Ограничения NOT NULL. Показывает, что столбец не может содержать *null*. Неоднократно использовалось в примерах в 4.3.

Ограничения уникальности. Ограничения уникальности гарантируют, что данные в определённом столбце или группе столбцов уникальны среди всех строк таблицы.

Ниже показан пример команды для создания ограничений уникальности (Используется укороченный пример про студентов, взятый выше):

```
-- Создает таблицу groups в схеме lab_schema
-- безымянное ограничение: значение name уникально
create table lab_schema.groups (
    id                bigserial primary key,
    name              varchar(30) not null unique,
```

```

        number_of_students integer not null check (number_of_students > 0),
    );

```

Первичный ключ. Ограничение первичного ключа означает, что образующий его столбец или группа столбцов может быть уникальным идентификатором строк в таблице. Для этого требуется, чтобы значения были одновременно уникальными и отличными от NULL. Таблица может иметь максимум один первичный ключ.

Помимо того варианта, в котором первичный ключ создавался в 4.3 (создание и удаление таблиц), возможно указать первичный ключ, как в примере ниже:

Внешний ключ. Ограничение внешнего ключа указывает, что значения столбца (или группы столбцов) должны соответствовать значениям в некоторой строке другой таблицы. Это называется *ссылочной целостностью* связанных таблиц.

Предположим, таблица *students* связывается с таблицей *groups* связью «многие-к-одному». Ниже показаны примеры команд, которые создают такое ограничение.

```

-- Создает таблицу groups в схеме lab_schema
create table lab_schema.groups (
    id                bigserial primary key,
    name              varchar(30) not null,
    number_of_students integer not null check (number_of_students > 0),
);

-- Создает таблицу students в схеме lab_schema с безымянным ограничением:
-- столбец id_groups - внешний ключ к таблице customer
create table lab_schema.students (
    id                bigserial primary key,
    first_name        varchar(30) not null,
    second_name       varchar(30) not null,
    middle_name       varchar(30),
    id_groups          bigint references lab_schema.groups(id),
    cell_phone        varchar(20),
    address            varchar(100),
    course            integer check (course > 0)
);

```

Если внешний ключ состоит из нескольких столбцов, число и типы столбцов в ограничении должны соответствовать числу и типам целевых столбцов.

Используя информацию выше, напомним и выполним запросы, создающие таблицы «*students*» и «*groups*» в инструменте запросов (код можно взять из примера в «создание и удаление таблиц») (Рис. 6) и убедимся, что данные таблицы успешно созданы (Рис. 7).

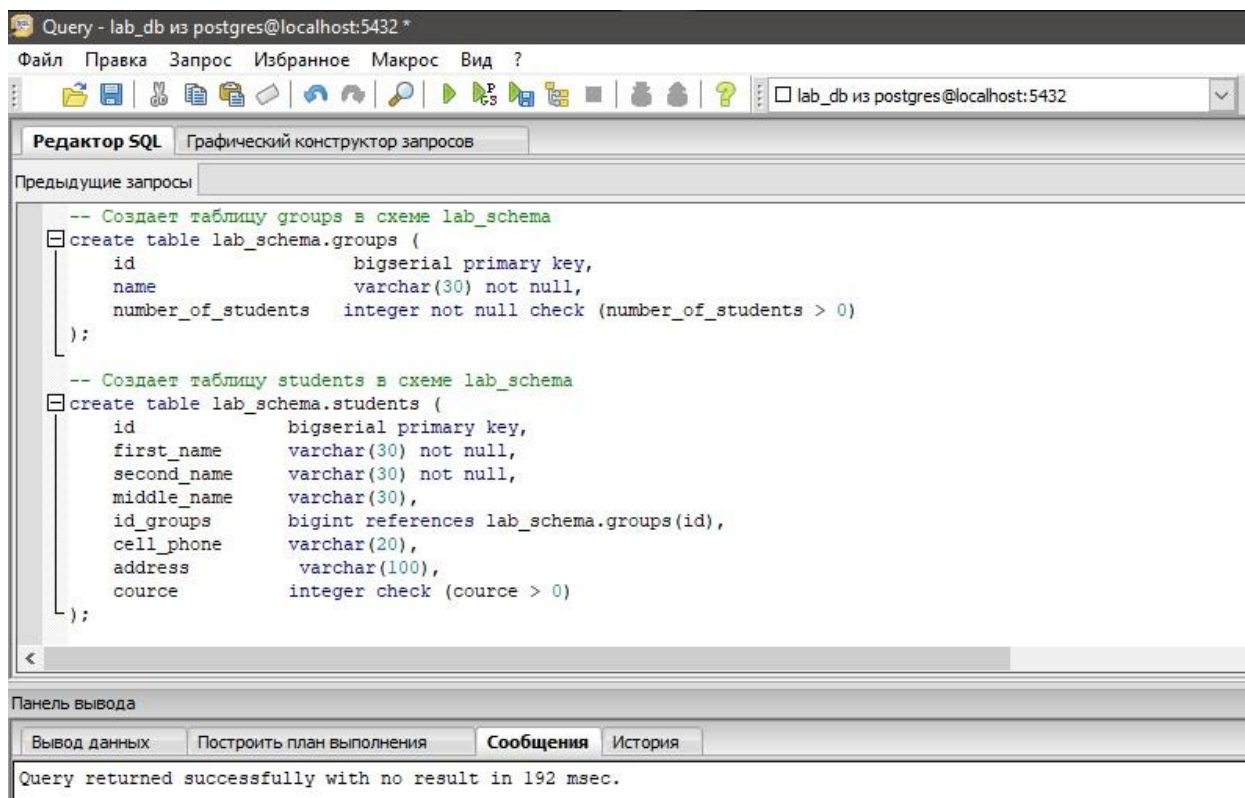


Рис. 6 – Окно «Редактор SQL» в инструменте запросов.

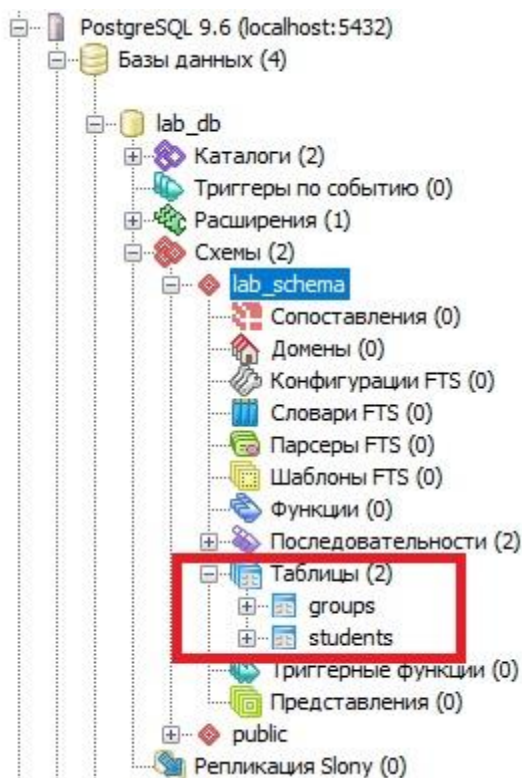


Рис. 7 – Список таблиц в бд lab_db

Остальные таблицы по структуре базы данных предлагается выполнить самостоятельно.

После создания всех таблиц, остается только выдать права на созданные объекты роли «*lab_test*».

Для того, чтобы управлять привилегиями роли, используются команды GRANT и REVOKE. С их помощью можно разрешать или запрещать:

- подключение к БД;
- выполнение чтения, записи, изменения, удаления данных;
- создание различных объектов в БД (таблиц, триггеров, внешних ключей, функций на определенном языке и т.д.);
- выполнение определенных функций (включая агрегатные).

Владелец объекта в БД по умолчанию имеет все права на него (хотя при необходимости может себя лишить их). Владелец объекта считается тот, кто его создал (позже владельца можно поменять).

Для выдачи всех прав на определенный класс объектов (например, на все таблицы в схеме) можно использовать вариант команды *GRANT ALL*. В этом случае важно помнить, что выдаются права на те объекты, которые существуют сейчас. Результат работы команды не касается будущих объектов (в этом примере – на те таблицы, которые позже будут добавлены в схему).

Важно также то, что все пользователи неявно имеют роль *public*. Если необходимо что-то разрешить или запретить всем пользователям (кроме суперюзеров, которым нельзя запрещать ничего, кроме подключения), то команды *GRANT* и *REVOKE* следует применять к роли *public*.

Подробнее о привилегиях, управление которыми доступно, можно прочитать в документации (стр. 1485; <https://postgrespro.ru/docs/postgresql/9.6/sql-grant>).

Ниже показаны примеры различных команд, связанных с управлением правами доступа в PostgreSQL.

```
-- Разрешить пользователю lab_test доступ к схеме lab_schema (не распространяется на объекты внутри неё)
```

```
grant usage on schema lab_schema to lab_test;
```

```
-- То же самое, что и команда выше, только пользователь lab_test сможет передать свое право другим пользователям (ролям)
```

```
grant usage on schema lab_schema to lab_test with grant option;
```

```
-- Разрешить пользователю lab_test создавать объекты в схеме test_schema
```

```
grant create on schema lab_schema to lab_test;
```

```
-- Разрешить пользователю lab_test все действия со всеми таблицами, которые сейчас существуют в схеме lab_schema
```

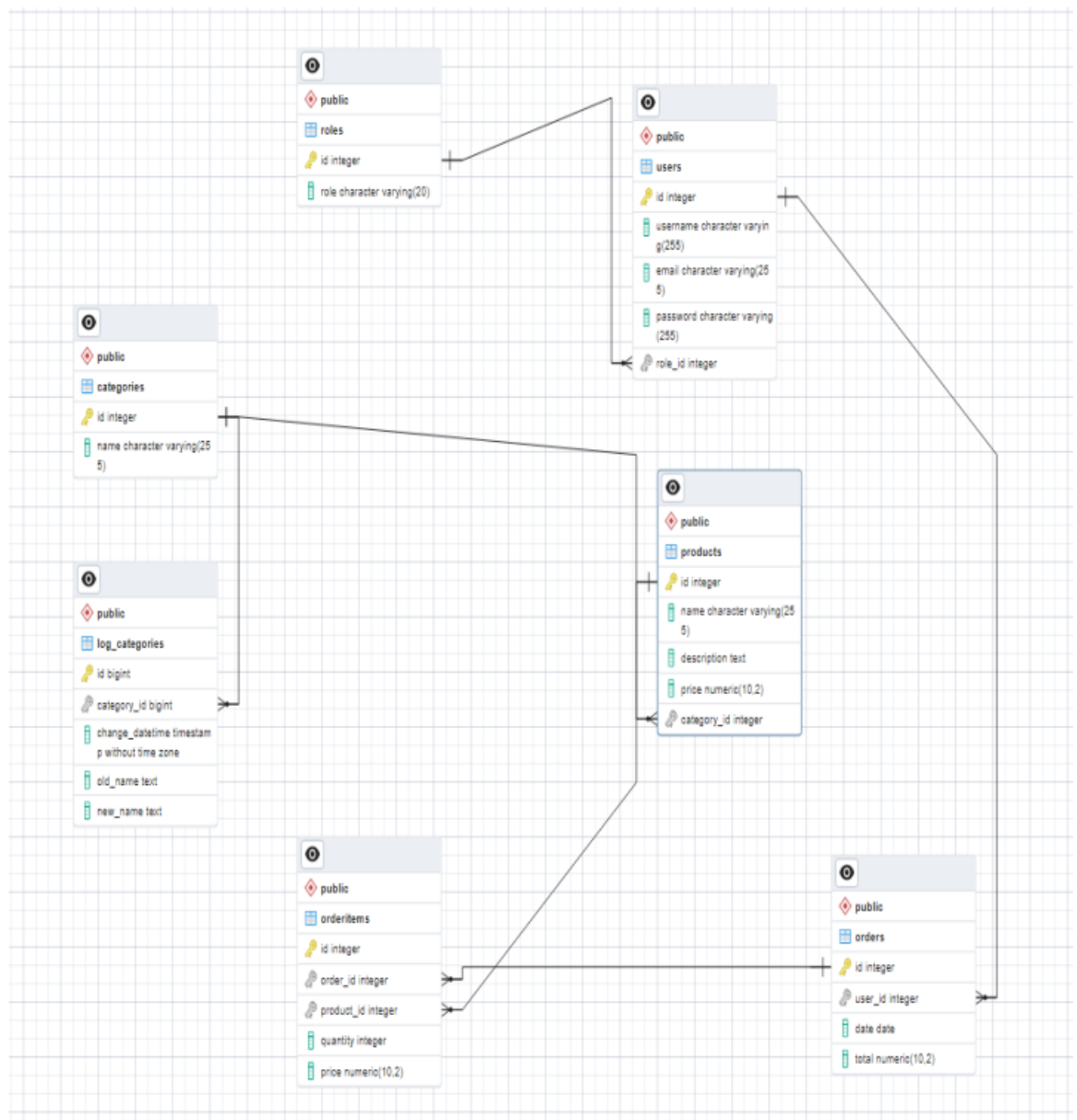
```
grant all on all tables in schema lab_schema to lab_test;
```

Разрешим пользователю «lab_test» все действия со всеми таблицами в схеме lab_schema, используя, PSQL Console (Рис. 8).

```
lab_db=# grant all on all tables in schema lab_schema to lab_test;  
GRANT
```

Рис. 8 – Разрешение пользователю «lab_test» всех действий со всеми таблицами в схеме «lab_schema».

Пример разработанной БД. ER-Диаграмма



Описание выбранной темы и краткое описание таблиц

Описание темы Выбранная тема для базы данных - интернет магазин.

Краткое описание таблиц

1. Таблица "categories" содержит категории товаров, которые продается в магазине. Включает в себя столбцы "id" - уникальный идентификатор категории, и "name" - название категории.

2. Таблица "orderitems" содержит информацию о каждом товаре, который был заказан в интернет-магазине. Включает в себя столбцы "id" - уникальный идентификатор элемента заказа, "orderid" - уникальный идентификатор заказа, "productid" - уникальный идентификатор товара, "quantity" - количество товара, заказанного пользователем, и "price" - цена каждого товара.

3. Таблица "orders" содержит информацию о каждом заказе, сделанном пользователями в магазине. Включает в себя столбцы "id" - уникальный идентификатор заказа, "userid" - уникальный идентификатор пользователя, который сделал заказ, "date" - дата заказа и "total" - общая стоимость заказа.

4. Таблица "products" содержит информацию о каждом продукте, который продается в магазине. Включает в себя столбцы "id" - уникальный идентификатор продукта, "name" - название продукта, "description" - описание продукта, "price" - цена продукта и "categoryid" - уникальный идентификатор категории, к которой относится данный продукт.

5. Таблица "users" содержит информацию о пользователях магазина. Включает в себя столбцы "id" - уникальный идентификатор пользователя, "username" - имя пользователя, "email" - адрес электронной почты пользователя, "pass word" - пароль пользователя и "roleid" - уникальный идентификатор роли пользователя.

6. Таблица "roles" содержит информацию о ролях пользователей магазина. Включает в себя столбцы "id" - уникальный идентификатор роли и "role" - название роли.